

Advanced Image Processing

Assignment 2 Report

Tejash More
SR: 27077

1 Graph-Based Image Segmentation using Normalized Cuts

1.1 Methodology

1.1.1 Graph Representation and Affinity Functions

Let an image be represented as a graph $G = (V, E)$, where V is the set of all pixels and E connects pairs of pixels. The weight w_{ij} measures the similarity between pixel i and pixel j . The original N-Cut affinity is defined as:

$$w_{ij} = \exp\left(-\frac{\|I(i) - I(j)\|^2}{\sigma_I^2}\right) \exp\left(-\frac{\|X(i) - X(j)\|^2}{\sigma_X^2}\right) \quad (1)$$

where $I(i)$ is the intensity, $X(i)$ is the spatial location, and σ_I, σ_X are scale parameters controlling tolerance.

To prevent the algorithm from bleeding across sharp physical boundaries, a modified edge-aware affinity incorporates the gradient magnitude E_{ij} :

$$w_{ij} = \exp\left(-\frac{\|I(i) - I(j)\|^2}{\sigma_I^2}\right) \exp\left(-\frac{\|X(i) - X(j)\|^2}{\sigma_X^2}\right) \exp\left(-\frac{E_{ij}^2}{\sigma_E^2}\right) \quad (2)$$

1.1.2 Normalized Cut Framework

To avoid the over-segmentation inherent in minimum cut algorithms, the Normalized Cut objective balances the cut cost with the total association of the regions:

$$Ncut(A, B) = \frac{Cut(A, B)}{Assoc(A, V)} + \frac{Cut(A, B)}{Assoc(B, V)} \quad (3)$$

Minimizing this function is computationally NP-hard. It is relaxed into a generalized eigenvalue problem:

$$Ly = \lambda Dy \quad (4)$$

where D is the diagonal degree matrix and $L = D - W$ is the unnormalized Graph Laplacian. The second smallest eigenvector (the Fiedler vector) provides the optimal continuous values for a two-way partition.

1.2 Experimentation

The experiments were conducted on a dataset of four images (Books, Chessboard, Kids, and Zebra). To accommodate the $\mathcal{O}(N^2)$ memory constraints of the dense affinity matrix W , the images were aggressively downsampled prior to feature extraction.

- **Part A:** Compared the original affinity against the modified edge-aware affinity using fixed parameters ($\sigma_I = 0.1, \sigma_X = 10.0, \sigma_E = 0.5$). Refer Fig: 1.
- **Part B:** Evaluated parameter sensitivity by varying $\sigma_I \in \{0.01, 0.1, 1.0\}$, and varying $\sigma_X \in \{2.0, 10.0, 50.0\}$, making the comparison in 9 different combinations of parameters. Refer Fig: ??.
- **Part C:** Executed multi-way segmentation ($K > 2$) comparing a localized recursive two-way approach against a global simultaneous approach utilizing K-Means on multiple eigenvectors. Refer Fig: 4

1.3 Results and Observations

1.3.1 Part A: Original vs. Modified Edge-Aware Affinity

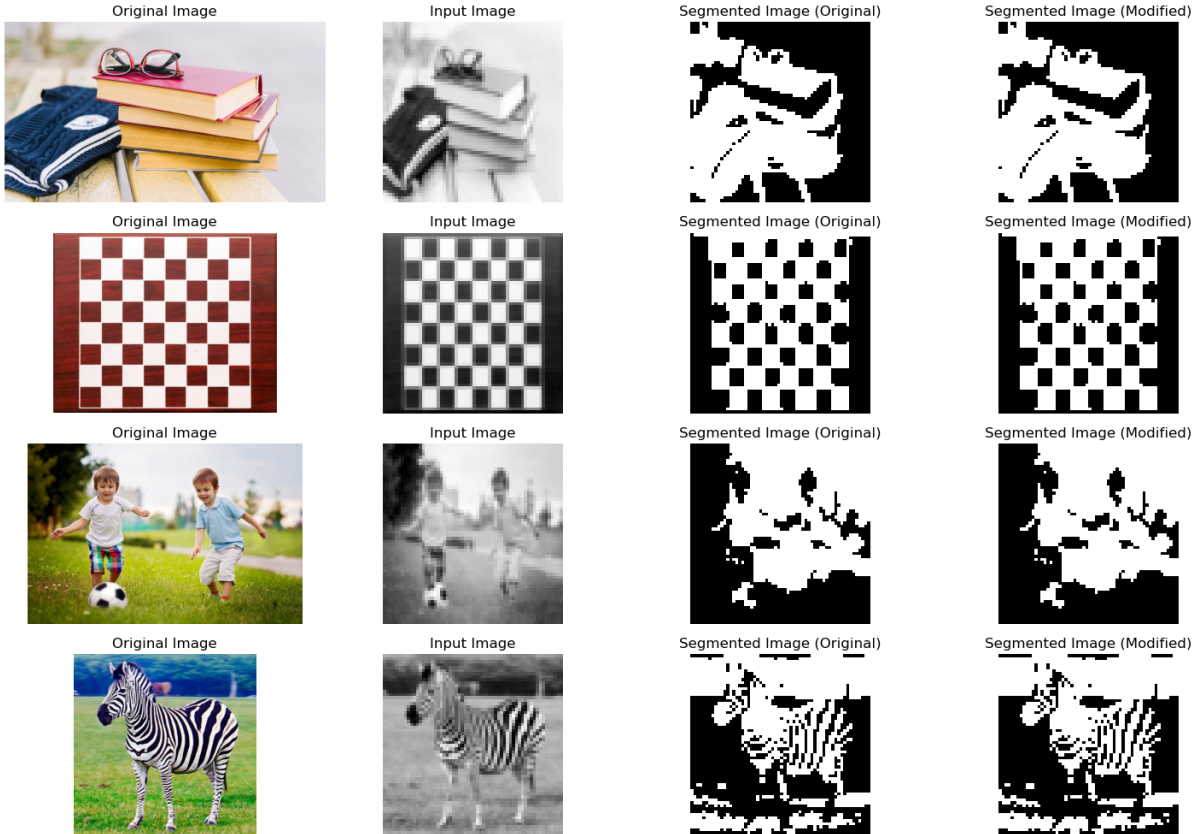


Figure 1: Comparison of Two-Way N-Cut using Original Affinity (Middle) vs. Modified Edge-Aware Affinity (Right).

Observation: The original affinity function relies strictly on pixel intensity similarity and spatial proximity. As observed in Figure 1, this causes the segmentation boundaries to occasionally smooth over sharp transitions, as spatially adjacent pixels are heavily

weighted even if they span an object’s boundary. By introducing the E_{ij} gradient penalty in the modified affinity, the exponential term $\exp(-E_{ij}^2/\sigma_E^2)$ acts as a severe mathematical barrier. When a high edge magnitude is detected, the affinity weight drops to near zero, forcing the Fiedler vector to segment precisely along the natural contours of the objects.

1.3.2 Part B: Parameter Analysis (σ_I and σ_X)

Observation:

- **Varying σ_I (Intensity Tolerance):** When σ_I is exceptionally low (e.g., 0.01), the algorithm becomes hyper-sensitive to minor intensity fluctuations, driving the affinity matrix toward a diagonal state and resulting in severe *over-segmentation* (the image shatters). Conversely, a high σ_I (e.g., 1.0) renders the algorithm effectively color-blind; it forgives large intensity differences, leading to *under-segmentation* where distinct objects are improperly fused.
- **Varying σ_X (Spatial Radius):** A low σ_X (e.g., 2.0) heavily penalizes spatial distance, restricting the algorithm to local area. This causes *over-segmentation* within large, uniform regions (like the background sky). A high σ_X (e.g., 50.0) creates a global network where distant pixels easily bond. This leads to *spatial bleeding* and *under-segmentation*, as disconnected objects sharing similar intensities are mathematically clustered together despite significant physical separation.



Figure 2: Effects of varying parameters (Top Half).

Seg Img Org (SI:0.1, SX: 10)



Seg Img Mod (SI:0.1, SX: 10)



Seg Img Org (SI:0.1, SX: 50)



Seg Img Mod (SI:0.1, SX: 50)



Seg Img Org (SI:1, SX: 2)



Seg Img Mod (SI:1, SX: 2)



Seg Img Org (SI:1, SX: 10)



Seg Img Mod (SI:1, SX: 10)



Seg Img Org (SI:1, SX: 50)



Seg Img Mod (SI:1, SX: 50)



Figure 3: Effects of varying parameters (Bottom Half).

1.3.3 Part C: Multi-Way Segmentation Strategies

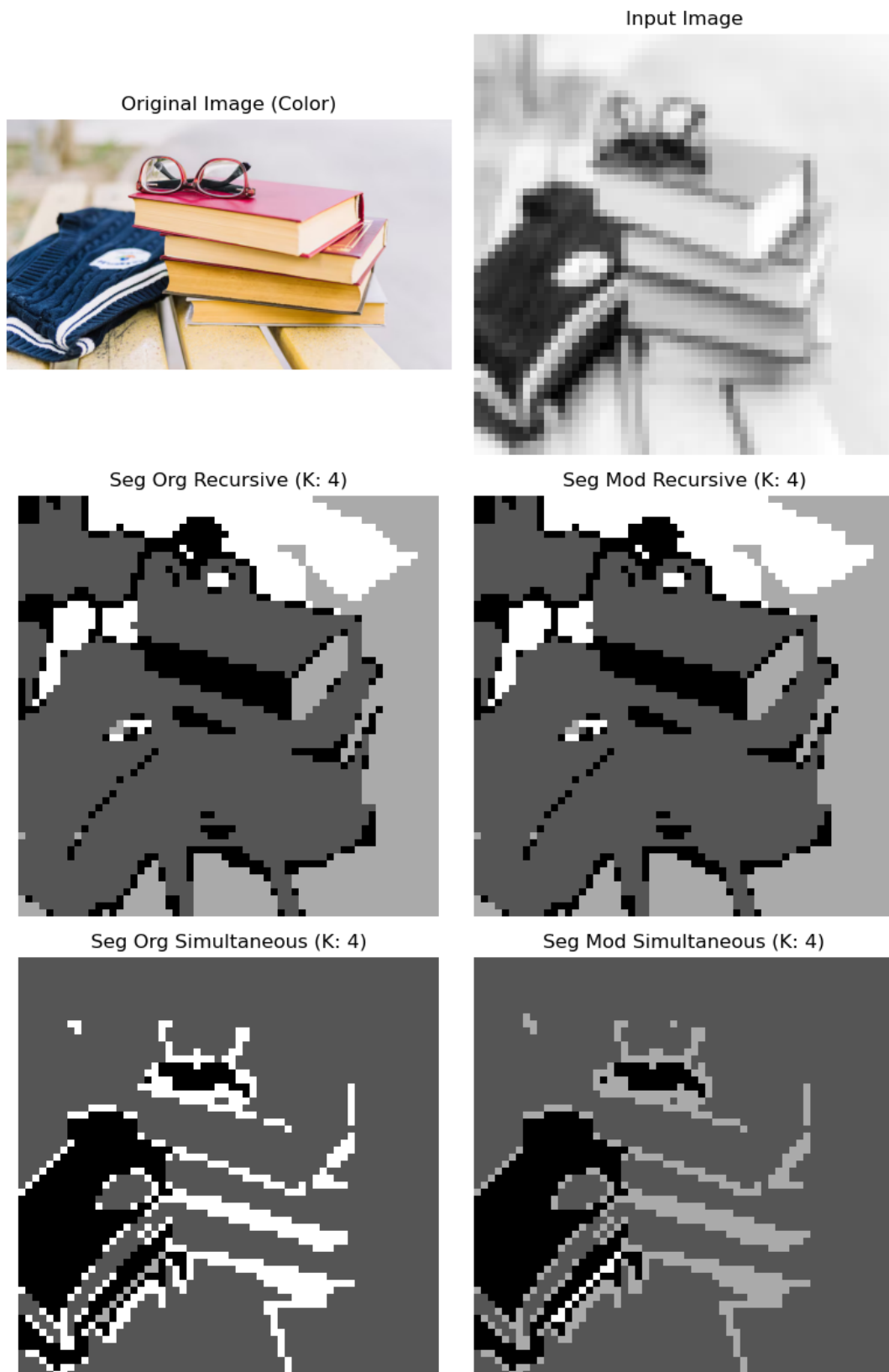


Figure 4: Simultaneous K-Way Partitioning vs. Recursive Two-Way Partitioning (Scaled to Grayscale).

Observation: Figure 4 highlights the fundamental mathematical differences between the two multi-way strategies:

- **Simultaneous Partitioning (Perceptual Quality):** By computing the first K eigenvectors once and applying K-Means clustering, this method maps the pixels into a multi-dimensional spectral coordinate space. It optimizes for global variance, resulting in visually smoother regions. However, it can sometimes perceptually fail by grouping physically disparate objects of the same intensity, as the global geometry overwhelms local spatial penalties.
- **Recursive Two-Way Partitioning (Region Consistency):** This strategy forces a forensic sub-audit of the image. By mathematically isolating sub-graphs and recalculating a new Laplacian (L_{sub}) at every step, it becomes highly sensitive to internal textural anomalies. While the visual output can appear more jagged compared to the simultaneous approach, it enforces strict local region consistency and often traces complex, high-frequency structural boundaries more accurately.

1.4 Conclusion

The Normalized Cut algorithm provides a robust, mathematically sound framework for perceptual grouping. The experiments demonstrate that an edge-aware affinity function is critical for proper boundary localization. Furthermore, parameter tuning (σ_I, σ_X) requires a delicate balance to avoid catastrophic over- or under-segmentation. Finally, the choice between Recursive and Simultaneous K-way partitioning acts as a trade-off between strict local region consistency and global perceptual smoothness.

2 Semantic Segmentation with Different Decoder Design

2.1 The Encoder: ResNet18

The encoder serves as the feature extractor. As the input image propagates through the residual blocks of ResNet18, spatial dimensions are progressively downsampled via strided convolutions and pooling, while the channel depth (feature complexity) increases. For an input image of size $H \times W \times 3$, the network generates intermediate feature maps at multiple scales.

- **Stem and Layer 1:**

- *Input:* $H \times W \times 3$
- *Output:* $\frac{H}{4} \times \frac{W}{4} \times 64$
- *Reasoning:* Initial strided convolutions and max-pooling rapidly reduce the spatial resolution to minimize computational overhead while extracting foundational low-level features (e.g., edges and colors).

- **Layer 2 (Depth 1 / x_2):**

- *Input:* $\frac{H}{4} \times \frac{W}{4} \times 64$

- *Output*: $\frac{H}{8} \times \frac{W}{8} \times 128$
- *Reasoning*: Extracts mid-level features. This tensor is explicitly saved in memory to serve as the high-resolution spatial reference for the final skip connection in the progressive decoder.
- **Layer 3 (Depth 2 / x_3):**
 - *Input*: $\frac{H}{8} \times \frac{W}{8} \times 128$
 - *Output*: $\frac{H}{16} \times \frac{W}{16} \times 256$
 - *Reasoning*: Extracts advanced morphological features. This tensor is saved to serve as the primary spatial reference for the first skip connection.
- **Layer 4 (Depth 3 / x_4):**
 - *Input*: $\frac{H}{16} \times \frac{W}{16} \times 256$
 - *Output*: $\frac{H}{32} \times \frac{W}{32} \times 512$
 - *Reasoning*: Represents the ultimate semantic bottleneck. The spatial resolution is heavily compressed (factor of 32), sacrificing precise geometric boundaries to achieve maximum semantic certainty regarding the presence of the target class.

We specifically extract features at three distinct depths, which are Depth (1, 2, 3):

2.1.1 Part (A): Single-Stage Decoder Design

The single-stage decoder attempts to reconstruct the full-resolution segmentation mask directly from the deepest, most highly compressed feature map (x_4). This requires a massive spatial expansion by a factor of 32 in a single mathematical operation.

We achieve this utilizing a Transposed Convolution layer. To map an $\frac{H}{32} \times \frac{W}{32}$ input back to $H \times W$, the layer is parameterized with a stride $S = 32$, a kernel size $K = 64$, and padding $P = 16$. The output spatial dimension O is governed by the equation:

$$O = (I - 1) \times S - 2P + K \quad (5)$$

- **Transposed Convolution:**

- *Input*: $\frac{H}{32} \times \frac{W}{32} \times 512$ (The x_4 tensor)
- *Output*: $H \times W \times 2$
- *Reasoning*: The 512 complex feature channels are mathematically crushed into exactly 2 channels, representing the probability logits for the "Background" and "Player" classes.

While mathematically viable, this architectural approach forces the network to blindly interpolate massive amounts of spatial geometry without any high-resolution reference points, intrinsically leading to a loss of boundary fidelity.

2.1.2 Part (B): Progressive Decoder with Skip Connections

To mitigate the spatial degradation inherent in the single-stage design, the progressive decoder reconstructs the image sequentially, integrating skip connections from earlier layers of the encoder.

The upsampling occurs in three distinct stages:

- **Stage 1 (First Upsampling & Merge):**

- *Upsample Input:* $\frac{H}{32} \times \frac{W}{32} \times 512$ (x_4)
- *Upsample Output:* $\frac{H}{16} \times \frac{W}{16} \times 256$ (Stride of 2 gently doubles the spatial resolution and halves the channel depth).
- *Concatenation:* The upsampled tensor is glued side-by-side along the channel axis with the x_3 skip connection ($\frac{H}{16} \times \frac{W}{16} \times 256$), resulting in an intermediate tensor of $\frac{H}{16} \times \frac{W}{16} \times 512$.
- *Blending Convolution:* A standard 3×3 convolution processes the concatenated tensor, mathematically fusing the deep semantic data with the sharp spatial reference, outputting a refined tensor of $\frac{H}{16} \times \frac{W}{16} \times 256$.

- **Stage 2 (Second Upsampling & Merge):**

- *Upsample Input:* $\frac{H}{16} \times \frac{W}{16} \times 256$ (Output of Stage 1)
- *Upsample Output:* $\frac{H}{8} \times \frac{W}{8} \times 128$ (Stride of 2 doubles resolution again).
- *Concatenation:* Glued with the x_2 skip connection ($\frac{H}{8} \times \frac{W}{8} \times 128$), resulting in a $\frac{H}{8} \times \frac{W}{8} \times 256$ tensor.
- *Blending Convolution:* Fuses the data and refines the boundaries further, outputting a tensor of $\frac{H}{8} \times \frac{W}{8} \times 128$.

- **Stage 3 (Final Resolution Recovery):**

- *Upsample Input:* $\frac{H}{8} \times \frac{W}{8} \times 128$ (Output of Stage 2)
- *Final Output:* $H \times W \times 2$
- *Reasoning:* With all high-frequency structural details recovered via the previous skip connections, a final transposed convolution (stride of 8) is applied to expand the intermediate $\frac{H}{8}$ resolution directly to the original $H \times W$ input size. The 128 channels are reduced to the final 2 class probability logits.

This architecture effectively routes the precise, high-frequency spatial details (edges, borders) preserved in the early layers of the encoder directly to the decoder, acting as a spatial guide for dense prediction.

2.2 Experimentation

2.2.1 Dataset and Preprocessing

The model was trained and evaluated on the Football Player Segmentation dataset. To facilitate batch processing through ResNet18, images and corresponding polygon annotations were dynamically processed into uniform spatial tensors.

- **Input Images:** Resized to 256×256 pixels and normalized using standard ImageNet statistics.
- **Ground Truth Masks:** The polygon coordinates were mathematically rendered onto a discrete 2D grid using `cv2.fillPoly`. Crucially, to match the image dimensions, the masks were resized to 256×256 strictly utilizing **Nearest Neighbor Interpolation**. This prevents fractional class labels (e.g., blending class 0 and class 1 to yield an invalid class 0.5) that occur with standard bilinear interpolation.

2.2.2 HyperParameters

Both architectures were optimized using the Adam optimizer (Adaptive Moment Estimation) to minimize the standard Cross-Entropy Loss over the spatial grid.

- **Batch Size:** 8
- **Epochs:** 15
- **Learning Rate:** 1×10^{-4}
- **Loss Function:** Cross-Entropy Loss Function

The models were evaluated utilizing stateful accumulators to calculate the Global dataset metrics at the conclusion of training, mathematically preventing the severe statistical bias associated with averaging individual batch IoU scores.

3 Results and Observations

3.0.1 Quantitative Evaluation

The quantitative performance of both architectures was measured over the test dataset utilizing Pixel Accuracy and Mean Intersection-over-Union (mIoU). The mIoU is calculated as:

$$mIoU = \frac{1}{C} \sum_{i=1}^C \frac{TP_i}{TP_i + FP_i + FN_i} \quad (6)$$

where C is the number of classes, TP represents true positives, FP false positives, and FN false negatives. The empirical results are documented in Table 1.

Table 1: Quantitative Comparison of Decoder Architectures

Architecture	Pixel Accuracy (%)	Global mIoU
Single-Stage Decoder	98.70	0.7063
Progressive Decoder	99.15	0.7992

3.0.2 Qualitative Analysis and Boundary Sharpness

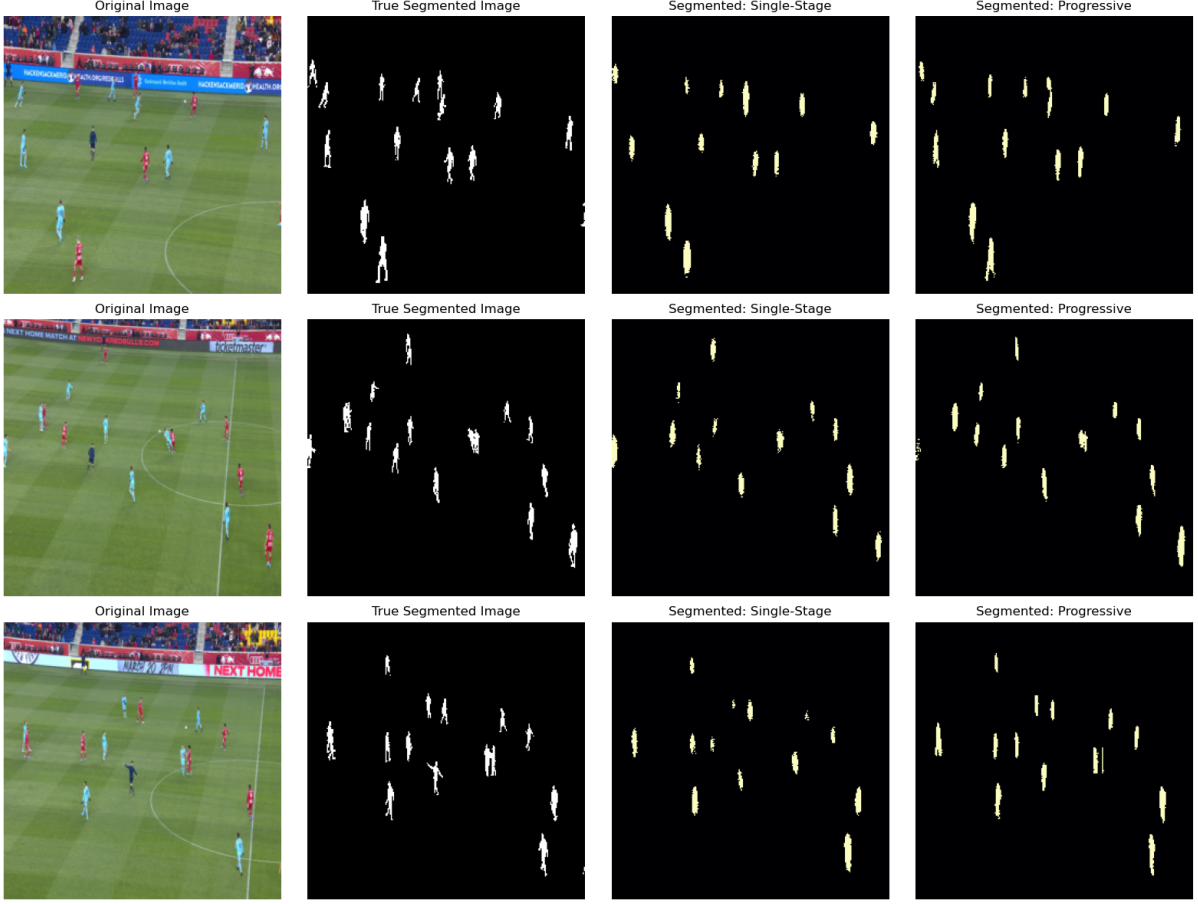


Figure 5: Qualitative comparison of predictions. Left: Original Image. Center-Left: Ground Truth Mask. Center-Right: Single-Stage Prediction. Right: Progressive Prediction.

Based on the qualitative visualizations presented in Figure 5, a clear difference is observable between the outputs of the two decoders.

- **Single-Stage Output:** The predictions generated by the single-stage decoder exhibit highly sparse and thin border edges. While the model successfully localizes the general mass of the football player, it completely fails to resolve fine details such as limbs, resulting in a generalized, blob-like mask.
- **Progressive Output:** The progressive decoder yields sharp, crisp boundaries that faithfully adhere to the anatomical contours of the player, matching the ground truth mask with high fidelity.

3.0.3 Discussion

Pixel accuracy is a flawed metric for segmentation because the majority of pixels in these images belong to the "Background" class. A model can achieve high accuracy simply by correctly classifying the large background and the center mass of the player, while failing at the edges. However, mIoU heavily penalizes false positives and false negatives at the object boundaries.

The single-stage upsampling fails because a standard convolution operates with local translation invariance. When the ResNet18 encoder crushes the 256×256 image into an $8 \times 8 \times 512$ feature map, it achieves semantic certainty (it knows a player is present) but suffers total spatial amnesia (it forgets exactly where the player’s arm mathematically ends). Expanding an 8×8 grid back to 256×256 via a massive stride forces the network to guess the spatial geometry, causing the jagged artifacts near the player’s boundary observed in Part A.

The skip connections introduced in Part B solve this algorithmic bottleneck. By passing the x_3 (16×16) and x_2 (32×32) feature maps directly to the decoder, we provide the network with high-frequency spatial tracing paper. The progressive upsampling allows the decoder to mathematically fuse the deep semantic knowledge of the x_4 layer with the crisp spatial geometry of the shallow layers at each resolution step. Consequently, the network learns to "color inside the lines," drastically reducing boundary errors and resulting in the significantly superior mIoU score.